Learning to Learn Othello: Rule-Based Bots, Classical Reinforcement Learning and Beyond

Palak Raisinghani

October 4, 2026

Abstract

Othello has simple rules, but its strategy is deep and often counter-intuitive. Moves that bring high immediate rewards can prove to be harmful later, and moves that seem to be of no great immediate value can prove extremely beneficial later on in the game. Othello is thus a strong benchmark to prove capabilty in learning delayed rewards. This project tests several different Rule Based and RL models on their ability to learn how to play Othello. Finally we develop a solid model for the game and offer an attractive interface to play the game against our RL-build Model that we call *Oroletto*.

Contents

1 Why Reinforcement Learning for Games **2**
- 1.1 Games as reinforcement-learning problems . . . 2
- 1.2 Landmark game-playing systems . . . 3
- 1.3 Why Othello . . . 3

2 Othello : The Board Game **4**
- 2.1 Board and pieces . . . 4
- 2.2 Rules . . . 4
- 2.3 Strategic concepts . . . 6
- 2.4 Board size and solved results . . . 6

3 Rule-Based Bots **6**
- 3.1 The random bot . . . 6
- 3.2 The heuristic bot . . . 7
- 3.3 The minimax bot . . . 7
  - 3.3.1 Choosing the search depth . . . 8
- 3.4 Head-to-head comparison . . . 9

4 Traditional Training Methods **11**
- 4.1 The Traditional RL Methods . . . 11
- 4.2 Common Experimental Setup for Training and Evaluation . . . 12
- 4.3 Results . . . 14
  - 4.3.1 Comparison against the three rule-based bots . . . 14
  - 4.3.2 Learning dynamics . . . 15
  - 4.3.3 Why the tabular methods fail . . . 16

4.3.4 Why linear function approximation plateaus . . . 17
4.3.5 Why DQN is the strongest, and still loses to search . . . 17
4.3.6 The disc-count ablation . . . 17

5 Self-Play Methods **18**
5.1 The methods . . . 18
5.2 Results . . . 20
5.3 Comparison with the solved game . . . 22
5.4 The final model: Oroletto . . . 23

6 Does Opponent Variety Help DQN? **23**
6.1 Design . . . 23
6.2 Results . . . 25

7 System Architecture **29**
7.1 Environment design . . . 29
7.2 Evaluation protocol . . . 30
7.3 Correctness validation . . . 30
7.4 Simulator throughput . . . 31
7.5 Playing in the terminal . . . 31

8 Next Steps **31**

A The Heuristic Bot: Weight Tuning **33**
A.1 Grid search . . . 33
A.2 Feature importance . . . 34

1 Why Reinforcement Learning for Games

1.1 Games as reinforcement-learning problems

Reinforcement learning studies agents that learn to act by trial and error, in order to maximise a cumulative reward [20]. They learn better policies while interacting with the environment. To do so they balance exploration and exploitation. This project looks at the classic board game of Othello and trains a RL model for the same. Historically, programmers developed rule-based bots which operated on handwritten rules.
For complicated games, like chess, this was hard and a tree search based approach was adopted. These bots were capable of choosing optimal actions at each step but were very computationally expensive.
With the advent of RL, games and especially board games found a new player. Board games offer two things that real-world RL problems rarely do:

- **A perfect, free simulator.** Rules are exact and games can be replayed millions of times, so data is limited only by compute.
- **Self-play.** An agent can play against copies of itself. It then always faces an opponent of its own strength, which produces an automatic curriculum without human data.

Games are also hard in a way that makes them useful benchmarks. The reward is *sparse* and *delayed*: a single number after dozens of moves. The learner must therefore solve the *credit-assignment problem*, which is working out which earlier moves caused the result. State spaces are also far too large to tabulate, so the agent has to *generalise* from positions it has seen to positions it has not.

1.2 Landmark game-playing systems

Table 1: Landmark game-playing systems. Not all of them use RL; Deep Blue is included as the classic *search-only* contrast.

Game	System	Year	Core idea
Checkers	Samuel's player [15]	1959	Learned a linear evaluation by comparing it with deeper search; an early ancestor of temporal-difference learning.
Othello	IAGO [13]	1982	Hand-built evaluation that introduced mobility as a central concept.
Backgammon	TD-Gammon [22]	1995	A neural network trained by TD(λ) through self-play; reached top human level.
Othello	Logistello [3, 4]	1997	Evaluation fitted by regression on pattern features from self-play games; beat the world champion 6–0.
Chess	Deep Blue [5]	1997	Massive alpha-beta search with a hand-tuned evaluation; no learning. Beat Kasparov.
Atari	DQN [12]	2015	Deep Q-learning from pixels, using experience replay and a target network.
Go	AlphaGo [17]	2016	Policy and value networks (supervised, then RL) guiding Monte Carlo tree search.
Go	AlphaGo Zero [18]	2017	Pure self-play from random play, with a single residual network plus MCTS.
Chess, shogi, Go	AlphaZero [19]	2018	The same algorithm and settings master all three games.
Dota 2	OpenAI Five [2]	2019	Large-scale PPO self-play in a team game with hidden information.
StarCraft II	AlphaStar [23]	2019	Multi-agent RL with a league of past and exploiter agents.
Atari, board games	MuZero [16]	2020	AlphaZero-style planning with a *learned* model of the rules.
Othello	Othello-GPT [11]	2023	A sequence model trained on move lists forms an internal board representation.

Table 1 shows a clear trajectory. Early systems learned only the *weights* of hand-designed features (Samuel, Logistello). TD-Gammon showed that a neural network could learn its own features from self-play. The AlphaGo line then combined learned networks with *search*: the network supplies intuition about which moves are promising and how good a position is, and tree search corrects the network's mistakes by looking ahead. Deep Blue is a reminder that search alone, with expert-tuned evaluation, is already very strong. This is why our ladder of rule-based bots (§3) includes a search bot.

1.3 Why Othello

Othello sits at a useful point between toy problems and grand challenges. The rules fit in three lines, but the state space is far beyond a lookup table even on 6×6. Its strategy is also counter-

intuitive: the disc-count trap (§2.3) means the obvious objective is the wrong one. This tests whether a learner discovers the real structure of the game or only the surface statistic. Finally, its small boards are solved, which gives ground truth that large games like Go do not have.

2 Othello : The Board Game

2.1 Board and pieces

Othello is a two-player, zero-sum, perfect-information game played on an $n \times n$ board, classically 8×8 [6]. The two opponents are assigned colours, black and white, and play with discs that are black on one side and white on the other. The game begins with two discs of each colour on the four centre squares, placed diagonally (Fig. 1a). The players then alternate moves, with Black moving first.[1] We name squares by column letter and row number, so on 6×6 the corners are a1, f1, a6 and f6. The rules do not depend on the board size, and this project uses 6×6, 8×8 and 10×10.

2.2 Rules

The rules below follow the formulation of [6].

1. **Legal moves.** A move places one additional disc of your colour on an empty square. It is legal only if the new disc *surrounds* one or more of the opponent's discs along a straight line, so that they lie between the new disc and another disc of your colour with no empty square in between. The line may run horizontally, vertically or diagonally. In other words, a move is legal only if at least one opponent disc is captured by it. On the very first move there are exactly four legal moves (Fig. 1a).

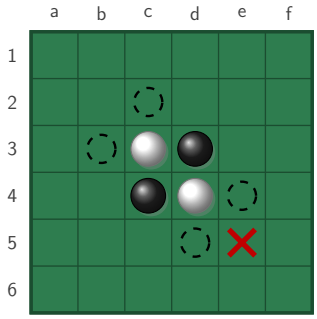
2. **Flipping.** Every surrounded opponent disc is replaced by a disc of your colour (in practice the disc is flipped over). This applies along *every* line the new disc closes at once, not just one. In Fig. 1b, Black plays c2, which surrounds the white disc on c3 against Black's disc on c4, so c3 becomes black; Fig. 1c–d shows one move capturing along 3 lines at once.

3. **Forfeiting a turn.** A player must move whenever they have a legal move; a turn can never be forfeited voluntarily. If a player has *no* legal move, their turn is forfeited (a *pass*) and the opponent moves again (Fig. 1e). A player may have to pass on several consecutive turns while the opponent keeps moving.

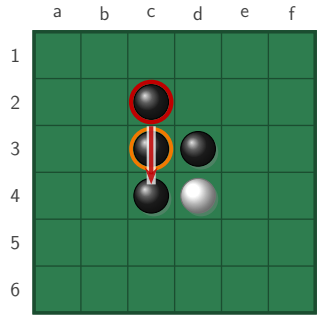
4. **End of the game.** The game ends when every square is filled, or when neither player can make a legal move.

5. **Winner.** When the game ends, the board is scored: the player with more discs of their colour wins (Fig. 1f). Equal counts are a draw.

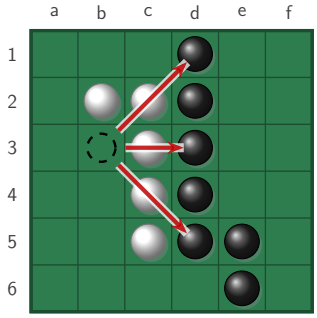
[1] The handout [6] lets White move first. The modern tournament convention, which this project and its code follow, is that Black moves first. The two differ only in which colour is called which.



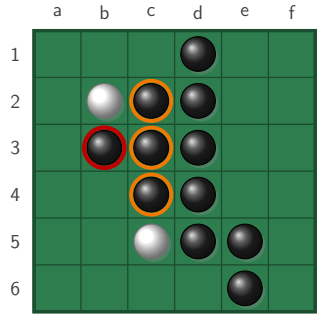
(a) **Start.** Two discs each, placed diagonally. Dashed circles: Black's four legal moves. ✗ e5 touches a white disc but surrounds nothing, so it is illegal.



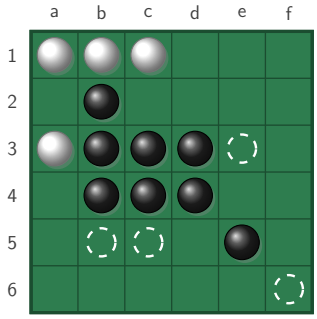
(b) **A capture.** Black plays c2 (red ring). The white disc on c3 lies between c2 and Black's c4 (arrow), so it flips (orange ring).



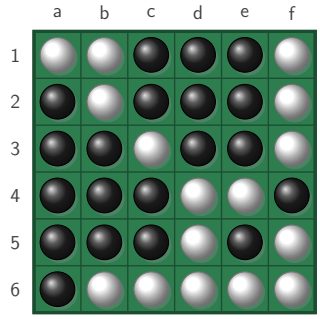
(c) **Several lines at once (before).** Black to move. Playing b3 (dashed) closes 3 lines at once; each arrow runs from the new disc to the black disc that anchors the line.



(d) **Several lines at once (after).** All 3 surrounded discs flip together, one per line.



(e) **A forced pass.** It is Black's turn, but no empty square surrounds a White disc, so Black must pass and White moves again (dashed: White's legal moves).



(f) **End of the game.** The board is full, so the game is over. Black has 20 discs and White 16: Black wins.

Figure 1: The rules of Othello on a 6×6 board. Every position comes from the game engine (`scripts/make_rule_diagrams.py`), so each one is legal and correctly flipped. Panels (a)–(b) are the same example as the interactive tutorial (`othello --tutorial`, §7.5).

2.3 Strategic concepts

Four ideas recur throughout this report. Each is defined here once. They also are the reason Othello is an attractive board game to experiment RL models on.

Stability and corners.
A disc is *stable* if it can never be flipped again. A corner disc is always stable, because no line can sandwich it. A stable corner also lets the discs along its edges become stable.

X- and C-squares.
The squares diagonally adjacent (X) and orthogonally adjacent (C) to a corner. Playing there while the corner is empty tends to give the corner to the opponent.

Mobility.
The number of legal moves a player has. Keeping your own mobility high and the opponent's low eventually forces the opponent onto X- and C-squares.

The disc-count trap.
Owning many discs in the middle game is usually a *weakness*. It flips discs the opponent can flip back, and it gives the opponent more moves. Only the final count matters, so maximising discs early is a classic beginner's error. This error turns out to matter for learning agents as well.

2.4 Board size and solved results

8×8 Othello has roughly 10^{28} legal positions and a game tree of about 10^{58} [1]. It was only recently weakly solved, as a draw under perfect play [21]. The 6×6 game was solved much earlier: it is a win for the second player (White), 16–20 [9]. We start on 6×6 because games are short and simulation is cheap. The board is still far too large to enumerate, so it remains a real learning problem.

3 Rule-Based Bots

Rule based bots are those that don't need any learning and have clear hand written rules. We use these as a benchmark for our RL models described later in the report. 3 rule based bots work on different general ideas and have been adapted to Othello for the purpose of this project. Every learning method is judged against fixed, rule-based opponents.We compare the performance of these three bots head to head, and we do so independently for each of three board sizes. In this section we use the shared evaluation protocol summarised below and detailed in §7.2.

Evaluation protocol (summary). Every match is played from K openings of six random plies, and each opening is played once with each colour. We report the *score* (win = 1, draw = $\frac{1}{2}$) with a 95% confidence interval, the win rate and the mean final disc margin.

3.1 The random bot

The random bot plays a uniformly random legal move. It is the floor of the ladder: any agent that has learned anything should beat it clearly. It also calibrates the game itself. Under random play the first mover scores slightly below half on every board size (Table 16), so evaluations must alternate colours.

3.2 The heuristic bot

The heuristic bot is *greedy*. For each legal move it evaluates the resulting position with a hand-written evaluation function and plays the best-scoring move. It looks exactly one ply ahead. The evaluation is a weighted sum of four features, each computed as "own minus opponent" and defined for any board size (Table 2):

- **Corners**: corner discs can never be flipped. They are *stable* and anchor the stable discs along their edges.
- **Mobility**: the number of legal moves available. High mobility means many options. Low opponent mobility forces the opponent into bad squares.
- **Disc count**: material on the board.
- **X-squares**: discs diagonally adjacent to an *empty* corner. Such a disc typically hands that corner to the opponent, so its weight is non-positive.

Table 2: The heuristic bot's evaluation features and the selected weights. These weights define the heuristic bot (and minimax's leaf evaluation) in every benchmark in this report. They were chosen by a grid search on 6×6 (Appendix A).

Feature	Definition	Weight
Corners	own − opponent corner discs	50
Mobility	own − opponent legal moves	5
Disc count	own − opponent discs	1
X-squares	own − opponent discs on an X-square next to an empty corner	-25

The weights were selected by a grid search over 96 candidate weightings, each played against three tuning opponents. Appendix A describes the search and gives the full results and a feature ablation. In short, corner occupancy carries most of the playing strength, mobility comes second, and disc count contributes almost nothing, which is the *disc-count trap* of §2.3.

The heuristic bot breaks ties at random and plays a uniformly random move with probability ε=0.05. It is therefore not one deterministic line that a tabular learner could memorise and exploit. Exploitability of that kind would give the learner a misleadingly high win rate. . which is the *easy* level of the terminal game.

3.3 The minimax bot

The minimax bot adds *lookahead*. It searches the game tree d plies deep, assuming both sides play their best, and scores the leaves with the tuned heuristic evaluation (or the exact result near the end of the game). Alpha-beta pruning [10] skips branches that cannot change the decision. Ordering moves statically (corners first, X- and C-squares last) makes the pruning more effective. Ties at the root are broken at random.

3.3.1 Choosing the search depth

d	Score (%)	W/D/L	Margin	Nodes/move	EBF	ms/game
1	56.4 ± 4.9	217/17/166	+2.3	4	–	3
2	73.5 ± 4.3	291/6/103	+9.9	16	3.7	7
3	86.1 ± 3.4	344/1/55	+17.2	68	4.3	23
4	88.2 ± 3.2	352/2/46	+17.3	166	2.4	56
5	**91.2 ± 2.8**	363/4/33	+20.5	614	3.7	210

Table 3: Alpha-beta minimax at depth d against the heuristic bot on 6×6 (200 paired openings = 400 games). Nodes per move and time per game were measured on 10 separate games. EBF: the ratio of nodes at depth d to nodes at $d-1$ (effective branching factor). Bold: best score.

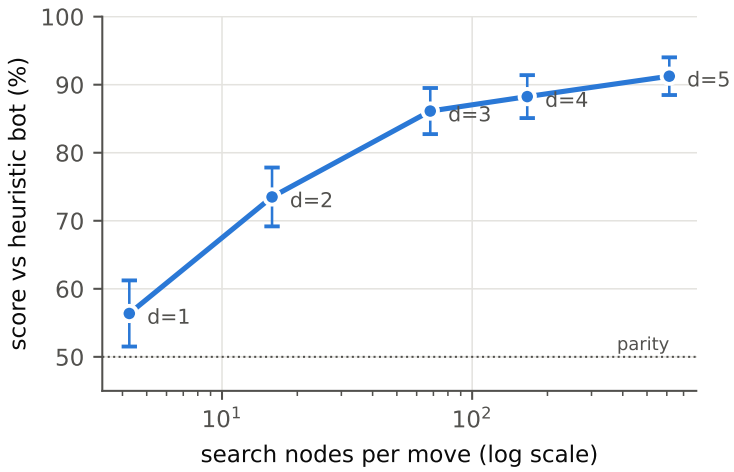


Figure 2: Strength vs. search cost for minimax depths 1–5 on 6×6. Error bars: 95% CI.

Table 3 and Fig. 2 show strength rising monotonically with depth, with clearly diminishing returns per node. The first two extra plies give most of the gain, and later plies each cost several times more search for a smaller improvement.

We keep **depth 4** as the reference minimax. It sits after the steep part of the curve, and depth 5 costs several times more per game. This matters because minimax will be evaluated thousands of times during RL training.

3.4 Head-to-head comparison

We compare these three bots against each other. As a result we obtain a strict strength ladder which is desirable since we want to use these bots as benchmarks later in the project. The opponents must form a clear strength ladder before any RL result can be interpreted. We set our requirements as follows :heuristic beat random in at least 90% of games and that minimax-d4 beat heuristic in at least 80%. The full round robin was played on all three board sizes with the weights for the heuristic bot tuned on 6×6.

Table 4: Diagnostic: win rate (%) and mean disc margin against the random bot for variants of the heuristic bot (6-ply random openings).

	6×6 (1000 games)		8×8 (400 games)	
Agent	Win	Margin	Win	Margin
heuristic ε=0.05	89.6	+17.5	91.5	+29.3
heuristic ε=0	90.8	+18.6	94.8	+31.3
minimax-d1	89.1	+17.5	95.2	+31.9
minimax-d2	**94.2**	+21.1	**99.8**	+38.9

Table 5: Diagnostic: heuristic vs. random on 6×6 as a function of the number of random opening plies (1000 games per row). "Distinct" counts the unique openings among the 500 generated.

		$\varepsilon = 0.05$		$\varepsilon = 0$	
Opening plies	Distinct	Win (%)	Score (%)	Win (%)	Score (%)
0	1	93.1	93.8 ± 1.5	94.1	94.5 ± 1.4
2	12	93.0	93.8 ± 1.5	92.7	93.5 ± 1.5
4	211	89.6	90.6 ± 1.8	91.1	92.0 ± 1.7
6	473	89.6	90.5 ± 1.8	90.8	91.6 ± 1.7

We tested three candidate causes for why Heuristc is not significantly (more than 95% better than random).

1. **Exploration noise.** Removing the ε-random moves helps only slightly (Tables 4 and 5), so it is not the main cause. As expected, the greedy bot and minimax-d1 perform the same, a second consistency check on the search code.

2. **Opening luck.** A 6×6 game lasts only about two dozen plies (Table 16), so six random opening plies are a substantial fraction of it. Table 5 supports this *partially*. Fewer random plies raise the win rate for heuristic , but only modestly, and the rate stays below 95%. 6 random plies occassionally gives the random bot an unfair advantage from which the heuristic bot cannot recover

3. **Myopia (horizon effect).** This is the remaining and, we infer, dominant cause. A one-ply evaluation cannot see the opponent's reply, so even a random opponent is sometimes handed a corner. Lookahead is the only change that substantially closes the gap: by overcoming a major set of blunders. (Table 4).

Table 6: Round robin between the reference opponents. Wins/draws/losses, win rate and score are from the first-named agent's perspective. Games = 2× openings. Gate: the revised threshold on *score* and its outcome.

Board	Match-up	Games	W/D/L	Win (%)	Score (%)	Margin	Gate
6×6	minimax-d4 vs heuristic	1000	870/10/120	87.0	87.5 ± 2.0	+17.2	≥80 PASS
	minimax-d4 vs random	1000	988/3/9	98.8	99.0 ± 0.6	+24.1	–
	heuristic vs random	1000	896/18/86	89.6	90.5 ± 1.8	+17.5	≥90 PASS
8×8	minimax-d4 vs heuristic	400	396/0/4	99.0	99.0 ± 1.0	+41.1	≥80 PASS
	minimax-d4 vs random	400	400/0/0	100.0	100.0 ± 0.0	+43.9	–
	heuristic vs random	400	366/7/27	91.5	92.4 ± 2.6	+29.3	≥90 PASS
10×10	minimax-d4 vs heuristic	200	199/0/1	99.5	99.5 ± 1.0	+61.6	≥80 PASS
	minimax-d4 vs random	200	200/0/0	100.0	100.0 ± 0.0	+68.6	–
	heuristic vs random	200	186/0/14	93.0	93.0 ± 3.5	+41.5	≥90 PASS

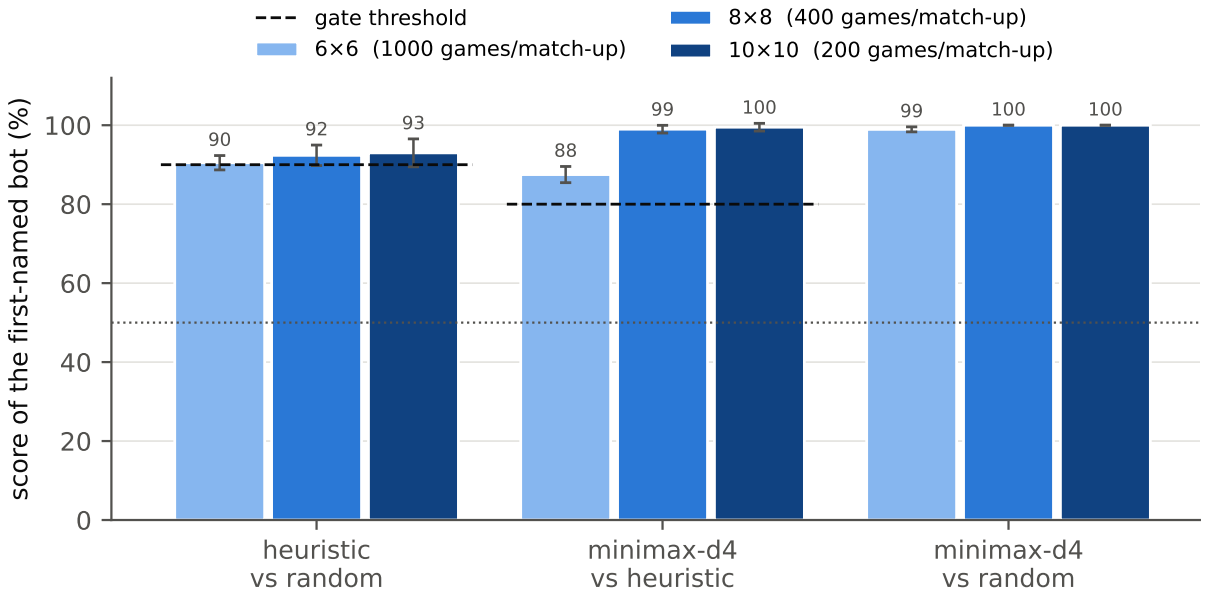


Figure 3: Head-to-head comparison of the three rule-based bots on each board size. Each bar is the score of the first-named (stronger) bot. Whiskers: 95% CI; dashed segments: gate thresholds; dotted line: 50% (parity). The 6×6-tuned weights of the heuristic were used on every size.

Fig. 3 and Table 6 confirm the intended ordering random < heuristic < minimax on every board size. The gap between minimax and heuristic widens as the board grows. We attribute this to two effects. Larger boards mean longer games with more decisions, so lookahead has more chances to pay off. And the weights were tuned on 6×6, which likely suits the greedy bot less on bigger boards, while search partly makes up for a mis-tuned evaluation.

4 Traditional Training Methods

In this section we look at five classical RL methods and train them on 6×6. All of them use the same fixed opponent and the same budget, and each is then compared against all three rule-based bots. The question is not only how well each method plays, but *why* it stops improving where it does.

4.1 The Traditional RL Methods

Tabular Q-learning [24]. Keeps a table entry $Q(s, a)$ for every state–action pair it has visited and updates it toward a bootstrapped one-step target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big].$$

The max makes it *off-policy*: it learns the value of the greedy policy while it behaves ε-greedily. To give the table its best chance, each position is stored in a canonical form under the 8 symmetries of the board, so symmetric positions share one entry. Unvisited entries read as 0.

SARSA [14]. Identical to Q-learning except that the target uses the action a_{t+1} the agent *actually* took next, i.e. $r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})$. It is therefore *on-policy*: it values the exploring policy itself, including its exploratory mistakes.

First-visit Monte Carlo control [20]. Does not bootstrap. After the game ends, every visited pair is moved toward the observed return G_t (here the final result) by a sample average, $Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s,a)}\,[G_t - Q(s, a)]$. Its targets are unbiased but have high *variance*: each one is a single noisy game outcome that depends on every later move by both players. (Positions never repeat within an Othello game, so first-visit and every-visit MC coincide.)

Linear function-approximation Q-learning. Replaces the table with a weighted sum of features, $Q(s, a) = \mathbf{w}^\top \boldsymbol{\phi}(s \oplus a)$, where $s \oplus a$ is the *afterstate*: the position right after the learner's move. The features are disc difference, mobility difference, corner difference, X-square difference, stable-edge-disc difference and a bias, each scaled to roughly unit range. The weights are learned by semi-gradient Q-learning, $\mathbf{w} \leftarrow \mathbf{w} + \alpha\,\delta_t\,\boldsymbol{\phi}$. Unlike a table, it *generalises*: every position with similar features gets a similar value, even if it has never been seen. But it can only represent evaluations that are linear in those features.

Vanilla DQN [12]. Replaces the table with a neural network Q_θ: an MLP with two hidden layers of 256 units over three input planes (own discs, opponent discs, legal moves). It outputs one value per square plus pass. Two components stabilise training: an *experience-replay* buffer (200k transitions, sampled uniformly), which breaks the correlation between consecutive positions, and a *target network* Q_{θ^-}, synced every 2,000 updates, which keeps the bootstrap target from chasing itself. The loss is $\text{Huber}\big(Q_\theta(s, a) - [r + \gamma \max_{a' \text{ legal}} Q_{\theta^-}(s', a')]\big)$. Illegal actions are masked both when acting and in the max. Unlike the linear model, the network can learn its own features from raw discs.

Disc-count ablation (linear FA). To test the disc-count trap (§2.3) on a learner, the linear method is trained three more ways: without the disc feature, with *only* the disc feature (plus bias), and with the full feature set but rewarded by the final *disc margin* instead of win/loss.

4.2 Common Experimental Setup for Training and Evaluation

All five methods (and the ablation variants) are trained and evaluated under exactly the same conditions. Only the learning algorithm differs.

Training.

Common opponent:
the heuristic bot of §3.2, with the tuned weights of Table 2 and $\varepsilon = 0.05$ random moves. It is the same fixed opponent for every method, run and game; nothing else is ever played during training.

Training algorithm:
the method being studied (§4.1): tabular Q-learning, SARSA, Monte Carlo control, linear function-approximation Q-learning or DQN. Every method runs inside one shared training loop (`othello/rl/common.py`), so they differ only in how they choose moves and update their values.

Learner's view (state):
the position at each of the learner's own decision points, from the side to move. The opponent's reply is part of the environment's transition from s_t to s_{t+1}.

Reward:
$+1$ for a win, 0 for a draw, -1 for a loss, given only at the end of the game; every other step gets 0. There is no discounting ($\gamma = 1$). (The disc-margin ablation replaces the final reward by the final disc margin.)

Exploration:
ε-greedy, with ε decaying linearly from 0.3 to 0.05 over the first half of training and held at 0.05 afterwards.

Step sizes:
tabular Q-learning and SARSA $\alpha = 0.1$; Monte Carlo the sample average $1/N(s, a)$; linear $\alpha = 0.01$; DQN as described in §4.1.

Games and openings:
200,000 training games per run. Each game starts from a random 6-ply opening drawn from a fixed pool of 20,000, and the learner alternates between Black and White every game.

Seeds:
3 independent runs (seeds 0, 1, 2) per method.

Evaluation.

What is evaluated:
the learner's *greedy* policy: no exploration ($\varepsilon = 0$), ties broken at random, and no learning during evaluation games.

Learning curves (during training):
every 5,000 training games the current policy plays 200 games (100 fixed openings, each with both colours) against the heuristic bot. The same openings are used at every checkpoint and for every method, so all the curves are directly comparable. Note that this opponent is the *training* opponent, so the curves measure progress against the bot each method is learning to beat.

Final comparison (after training):
the final policy plays all three rule-based bots: 1,000 games against random, 1,000 against heuristic and 500 against minimax-d4 (fewer, because each minimax move costs a 4-ply search). These games use a *different* fixed set of random 6-ply openings from the learning curves, shared by every method and seed.

Why three opponents:
each answers a different question. Random asks whether the method plays sensible Othello at all. Heuristic asks how well it beats the opponent it trained against. Minimax-d4 asks how it fares against a much stronger player it never saw. Real playing skill should show up against all three, roughly in that order. Beating the training opponent while failing against the others signals that the method has exploited that one opponent rather than learned the game.

Metric:
the *score* $= (\text{wins} + \frac{1}{2}\,\text{draws})/\text{games}$, from the learner's side. Each colour is played equally often, so the first-move advantage cancels. Win rate and mean disc margin are also recorded.

Uncertainty:
each final match of 1,000 games has a 95% confidence interval of about ± 3 percentage points (about ± 4 at 500 games). Across the 3 seeds we report mean $\pm$ standard deviation, and plots show the min–max range across seeds as whiskers or bands.

Extra diagnostics:
for the tabular methods, the fraction of evaluation positions already present in the table (*coverage*) and the variance of their update targets. For the linear method, the learned feature weights.

4.3 Results

4.3.1 Comparison against the three rule-based bots

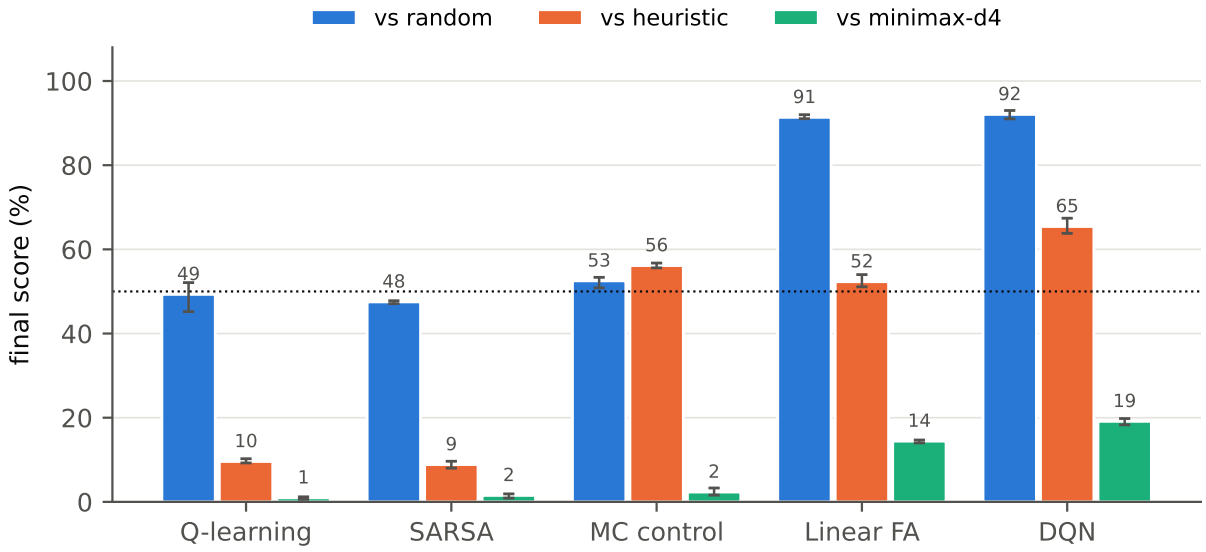


Figure 4: Final score of each traditional method against the three rule-based bots on 6×6. Bars: mean over 3 seeds; whiskers: min–max across seeds; dotted line: 50% (parity). Against random and heuristic each bar is 1,000 games per seed; against minimax-d4 it is 500.

Table 7: Final evaluation of the traditional methods (mean ± std over seeds, %). "Win vs heur." is the raw win rate against the training opponent. "Train" is the wall-clock hours per run on one CPU core. The reference row is the heuristic bot's own result from the rule-based round robin (Table 6). Bold: best per column.

		Score vs. (%)				
Method	Seeds	random	heuristic	minimax-d4	Win vs heur.	Train (h)
Q-learning	3	49.3 ± 3.6	9.6 ± 0.6	1.0 ± 0.3	8.5	0.5
SARSA	3	47.5 ± 0.4	8.9 ± 0.8	1.5 ± 0.6	7.7	0.4
MC control	3	52.5 ± 1.4	56.1 ± 0.6	2.3 ± 0.9	54.8	0.3
Linear FA	3	91.4 ± 0.5	52.3 ± 1.5	14.4 ± 0.4	51.3	0.8
DQN	3	**92.1 ± 1.0**	**65.4 ± 1.8**	**19.1 ± 0.8**	64.0	1.5
Heuristic bot (reference)	–	90.5	–	12.5	–	–

Figure 4 and Table 7 separate the methods into three tiers. The tabular TD methods (Q-learning and SARSA) never leave random play: they draw level with the random bot and lose almost every game to the other two. Linear function approximation reaches the strength of the heuristic bot: it scores about as well as the heuristic does against random, breaks even against the heuristic, and loses to minimax-d4 by a similar margin. DQN is the strongest method on *all three* opponents, and is the only one that clearly beats its training opponent while still dominating random play.

Monte Carlo control does not fit this ordering. It beats the heuristic bot it was trained against,

yet against the *random* bot it does no better than a coin flip. A policy that had learned to play Othello would find the random bot the easiest of the three opponents. MC control instead does well only against the one opponent it has seen. We call this *overfitting to the training opponent*: a policy that exploits the specific habits of one opponent instead of learning the game. Evaluating against the training opponent alone would have ranked MC control among the best methods. The three-bot comparison is what exposes it. No method comes close to minimax-d4. Every one of them chooses moves without lookahead, and none overcomes the horizon effect that already limited the one-ply heuristic bot (§3).

4.3.2 Learning dynamics

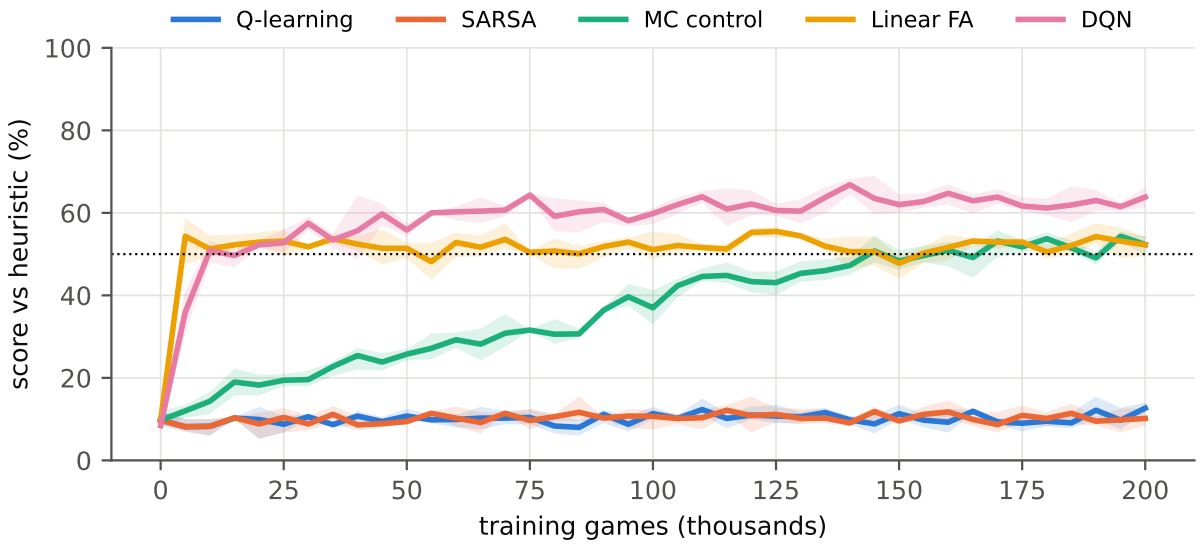


Figure 5: Learning curves: greedy-policy score against the heuristic bot (200 games per point) during training. Lines: mean over 3 seeds; bands: min–max. Q-learning and SARSA overlap almost exactly.

The curves in Fig. 5 show three different failure modes.

- **Flat at the bottom (Q-learning, SARSA).** No measurable improvement over the whole budget.
- **Immediate plateau (linear FA).** Reaches parity within the first evaluation interval and then does not move. Additional data does not help.
- **Slow, steady climb (MC control, DQN).** Both are still rising at the end of the budget, MC control from a much lower start.

4.3.3 Why the tabular methods fail

Table 8: Tabular diagnostics at the end of training (mean over 3 seeds). Coverage is the fraction of the learner's decision positions during the final evaluation games that already have an entry in its table. Update-error variance is the variance of (target $- Q$) over the updates of the first and of the last 5,000 training games.

		Coverage (%)		Update-error var.	
Method	States (10^6)	vs heuristic	vs random	first 5k	last 5k
Q-learning	1.92	36.4	8.2	0.066	0.057
SARSA	1.92	36.6	8.3	0.066	0.058
MC control	1.52	66.9	9.6	0.367	0.622

The curse of dimensionality. Table 8 shows the mechanism directly. After 200,000 games each table holds between one and two million canonical positions, yet in games against the random bot fewer than one in ten of the positions the learner meets are in its table. A lookup table has no notion of similarity between positions, so a position it has not seen is a position it knows nothing about, and it falls back to an arbitrary choice. This is the *curse of dimensionality* in its plainest form: the number of reachable positions is so large that a table cannot be filled from any feasible amount of play. It is the same even after merging the 8 board symmetries. Coverage is markedly higher against the heuristic bot, because a mostly deterministic opponent keeps leading the game back into the same lines. That difference is what MC control exploits.

Why Q-learning and SARSA are indistinguishable. The two update rules differ only in which next-state entry they bootstrap from: the greedy one or the one actually taken. A unit test confirms they produce different targets when the next state's entries exist (`tests/test_rl.py`). In practice the next state is usually unvisited, so both entries read the default value of zero, the two targets coincide and the learners behave the same. We attribute the TD methods' failure to *bootstrapping* under sparse revisits. A one-step target $r + \gamma Q(s', \cdot)$ only carries information backwards one move per visit. The final reward therefore has to travel back through many revisits of the same line before it reaches early positions, and almost no line is revisited that often.

Why MC control behaves differently. MC control does not bootstrap. After every game it moves *all* visited entries straight toward the actual outcome, so a single winning game against the heuristic bot marks every position on that line as good at once. Against a nearly deterministic opponent, starting from a bounded set of openings, the same lines come back often enough for these marks to accumulate. Its coverage against the heuristic bot rises together with its score. It is effectively memorising a growing book of winning lines against *that one bot*. Coverage against the random bot stays low, which is why the memorised book is useless there. The price of not bootstrapping is *variance*. Each MC target is a single, noisy game outcome, and its update-error variance is several times that of the TD methods. That variance is the bias–variance trade-off in action: unbiased targets, but slow and noisy progress, consistent with its slow climb in Fig. 5.

Design note. We added the heuristic bot's ε-randomness and the random openings specifically to stop a tabular learner from memorising lines against it (§3). This was *not sufficient*. MC control still found enough repetition to exploit. Any future result measured only against the training opponent should be treated with suspicion. The curve was still rising at the end of the budget, and we expect, but have not tested, that longer training would increase this exploitation without improving play against other opponents.

4.3.4 Why linear function approximation plateaus

The linear learner *generalises*: two positions with similar features get similar values whether or not either was seen. That is why it beats the random bot as convincingly as the heuristic bot does, after almost no training. But it reaches its ceiling almost immediately and stays there. Its features are essentially the heuristic bot's own features plus edge stability, so the best policy it can represent is a re-weighted one-ply heuristic. Learning finds those weights within a few thousand games, and the learned weights reproduce the ordering found by the weight search (Appendix A): corners first, then mobility, with disc count near zero (Table 9). This is a *capacity* or *inductive-bias* limit rather than a data limit. The model class itself contains nothing better than the heuristic, so more games cannot help.

4.3.5 Why DQN is the strongest, and still loses to search

DQN combines the two missing ingredients. It generalises across positions like the linear model, but it learns its own features from raw disc planes instead of being restricted to hand-chosen ones. Replay lets each game be learned from many times, and the target network keeps the bootstrapped targets stable while it does. The result is the only method that improves on the heuristic bot without losing generality. It still chooses each move from a single forward pass, with no lookahead. Minimax-d4 punishes exactly the tactical oversights a one-ply policy cannot see, which is why DQN loses heavily to it. Its curve is still rising, so a longer run would likely help somewhat. However, closing the gap to search is the job of the self-play and MCTS methods in §8, not of more DQN training.

4.3.6 The disc-count ablation

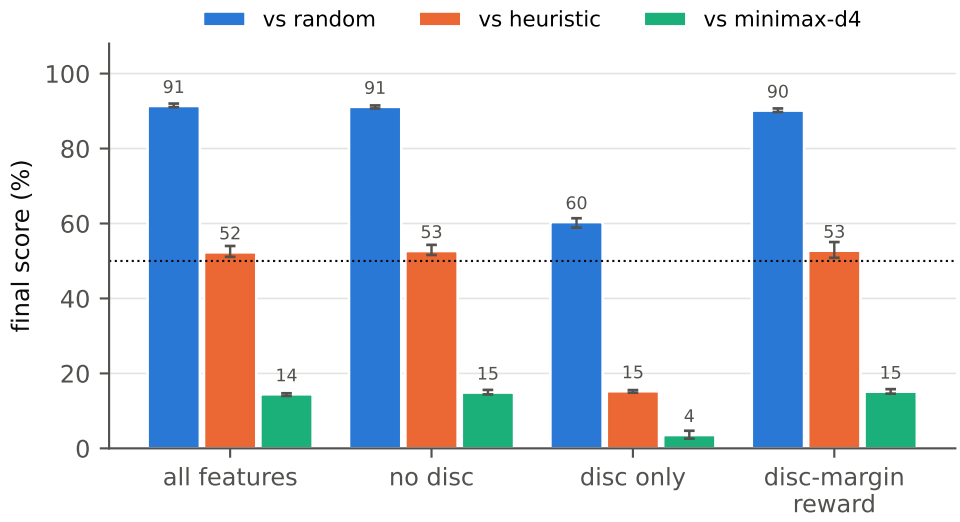


Figure 6: Disc-count ablation for linear function approximation. Same format as Fig. 4.

Table 9: Disc-count ablation (mean ± std over 3 seeds, %), with the learned weights (mean over seeds) on the scaled disc, mobility and corner features.

	Score vs. (%)			Learned weight		
Variant	random	heuristic	minimax-d4	disc	mob.	corner
All features, win/loss reward	91.4 ± 0.5	52.3 ± 1.5	14.4 ± 0.4	-0.05	+0.40	+0.86
No disc feature	91.1 ± 0.4	52.7 ± 1.4	14.9 ± 0.6	–	+0.44	+0.88
Disc feature only	60.4 ± 1.3	15.2 ± 0.4	3.6 ± 1.1	+0.87	–	–
All features, disc-margin reward	90.1 ± 0.5	52.8 ± 2.1	15.1 ± 0.6	+0.00	+0.26	+0.56

Fig. 6 and Table 9 separate two versions of the disc-count trap.

- **Disc-count *features*: the trap is reproduced.** Removing the disc feature changes nothing, and the full model learns to give it essentially zero weight. A learner restricted to disc count *only* collapses to the strength of the disc-greedy bot from the tuning study (Appendix A), barely above random. Disc count simply does not carry the information needed to play well. The learner rediscovers what the rule-based tuning found (Appendix A).
- **Disc-count *reward*: no measurable harm.** Rewarding the final disc margin instead of the win leaves strength unchanged on all three opponents. This *contradicts* the expectation in our original plan. We attribute it to the features. With corner and mobility features available, the positions that maximise the final margin are the same ones that maximise the chance of winning, so the two objectives agree. The learned weights only shrink in scale, as the smaller reward range would predict, and keep their ordering. The trap lies in valuing discs *during* the game, which the features control, not in scoring them at the *end*, which is all the reward does.

5 Self-Play Methods

Every method in §4 learned by playing the heuristic bot, so it could at best learn to beat *that* bot. In *self-play* the learner plays against itself, or against frozen copies of its earlier selves. The opponent therefore improves as the learner does, and no hand-written opponent is needed during training. This section compares three self-play methods on 6×6. The evaluation is unchanged: the same three rule-based bots, openings and protocol as before. The heuristic bot is now used *only* for evaluation and is never trained against. The strongest agent becomes the project's final model, *Oroletto*.

5.1 The methods

DQN+ (Double DQN with an opponent pool). The DQN of §4 with two changes. First, the bootstrap target is a *Double DQN* target [8], $r + \gamma\, Q_{\theta^-}\big(s', \arg\max_{a'} Q_\theta(s', a')\big)$: the online network picks the next action and the target network scores it. This removes the systematic overestimation that comes from taking a max over noisy estimates. Second, the opponent is drawn from an *opponent pool*. Every 5,000 games a frozen snapshot of the learner joins the pool (at most 20 kept), and each training game is played against the live learner with probability $\frac{1}{2}$, otherwise against a random snapshot. Playing past selves protects against *strategy cycling*, where a learner that only ever plays its latest self forgets how to beat strategies it has already moved past.

Self-play PPO. Proximal Policy Optimisation [7] learns a policy $\pi_\theta(a \mid s)$ and a value $V_\theta(s)$ directly. Here an MLP with a shared trunk, a masked softmax over legal moves and a tanh value head is used. Experience is collected from complete games, 4,096 learner decisions per update, with advantages $\hat{A}_t$ from generalised advantage estimation ($\lambda = 0.95$). The policy is improved with the *clipped* objective

$$\mathcal{L}^{\text{clip}} = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \qquad \rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)},$$

with $\epsilon = 0.2$. Clipping keeps each update close to the policy that collected the data, which is what makes reusing a batch for several epochs safe. PPO uses the same opponent pool as DQN+, and only the learner's own moves are used for updates, so the data stay on-policy.

AlphaZero. Following [19], a single residual network $f_\theta(s) = (\mathbf{p}, v)$ outputs a move distribution $\mathbf{p}$ and a value $v \in [-1, 1]$. It has 4 residual blocks of 64 channels (about 0.3 M parameters), and its input planes are own discs, opponent discs and legal moves. Moves are chosen by *Monte Carlo tree search* (MCTS) guided by the network. Each of the 64 simulations per move descends the tree by the PUCT rule

$$a^* = \arg\max_a \Big[\, Q(s,a) + c_{\text{puct}}\, P(s,a)\, \frac{\sqrt{N(s)}}{1 + N(s,a)}\Big],$$

where Q is the mean value of the simulations through (s, a), P the network's prior, N visit counts and $c_{\text{puct}} = 1.5$. The leaf is evaluated by the network rather than by a random rollout. In self-play, Dirichlet noise is mixed into the root priors, and the first 10 plies are sampled in proportion to visit counts to diversify the games; after that the most-visited move is played. Each position stores its visit distribution $\boldsymbol{\pi}$ and the final result z. The network is trained on a sliding window of the latest 150,000 positions to minimise

$$\mathcal{L} = (z - v)^2 \ - \ \boldsymbol{\pi}^\top \log \mathbf{p} \ + \ c\,\|\theta\|^2,$$

and each batch is transformed by a random one of the 8 board symmetries. The key idea is that *search acts as a policy-improvement operator*. MCTS with the current network plays better than the network alone, and the network is then trained to imitate the search. The next search starts from a better prior, and so on.

Budgets. DQN+ and PPO play 200,000 self-play games, like the classical methods. AlphaZero plays 100,000. One AlphaZero game costs far more compute, because every move runs 64 network-guided simulations. Each method uses 3 seeds. Wall-clock times are reported alongside game counts.

5.2 Results

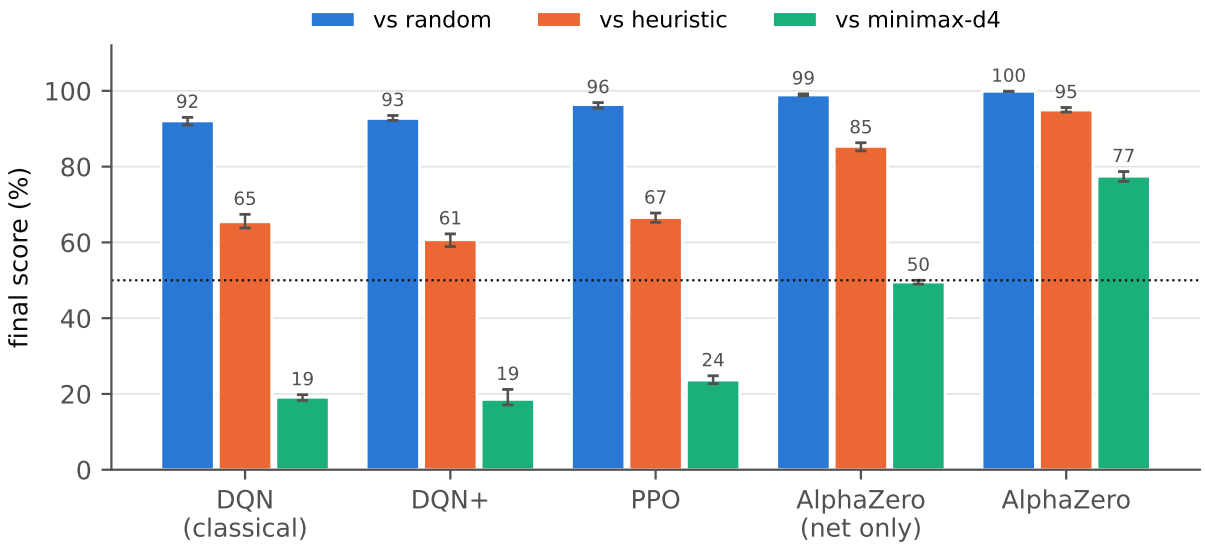


Figure 7: Final score of the self-play methods against the three rule-based bots on 6×6, with classical DQN (§4) for reference. Bars: mean over 3 seeds; whiskers: min–max; dotted line: 50%. "AlphaZero (net only)" plays the trained network's top move with no search; "AlphaZero" searches 64 simulations per move.

Table 10: Self-play methods compared (mean ± std over 3 seeds). "Games to 60/80%" is the number of training games after which the score against the heuristic bot first reaches that level (mean over seeds; evaluated every 5,000 games, so ≤5k means "by the first checkpoint"; – means never). Train: wall-clock hours per run.

		Final score vs. (%)			Games to			
Method	Seeds	random	heuristic	minimax-d4	60%	80%	Games	Train (h)
DQN (classical)	3	92.1 ± 1.0	65.4 ± 1.8	19.1 ± 0.8	43k	–	200k	1.5
DQN+	3	92.7 ± 0.7	60.7 ± 1.7	18.6 ± 2.3	97k	–	200k	1.3
PPO	3	96.4 ± 0.9	66.6 ± 1.2	23.7 ± 1.1	58k	–	200k	1.0
AlphaZero (net only)	3	98.9 ± 0.3	85.4 ± 1.1	49.5 ± 0.5	≤5k	15k	100k	
AlphaZero	3	**99.9 ± 0.0**	**94.9 ± 0.6**	**77.5 ± 1.3**	≤5k	≤5k	100k	7.8

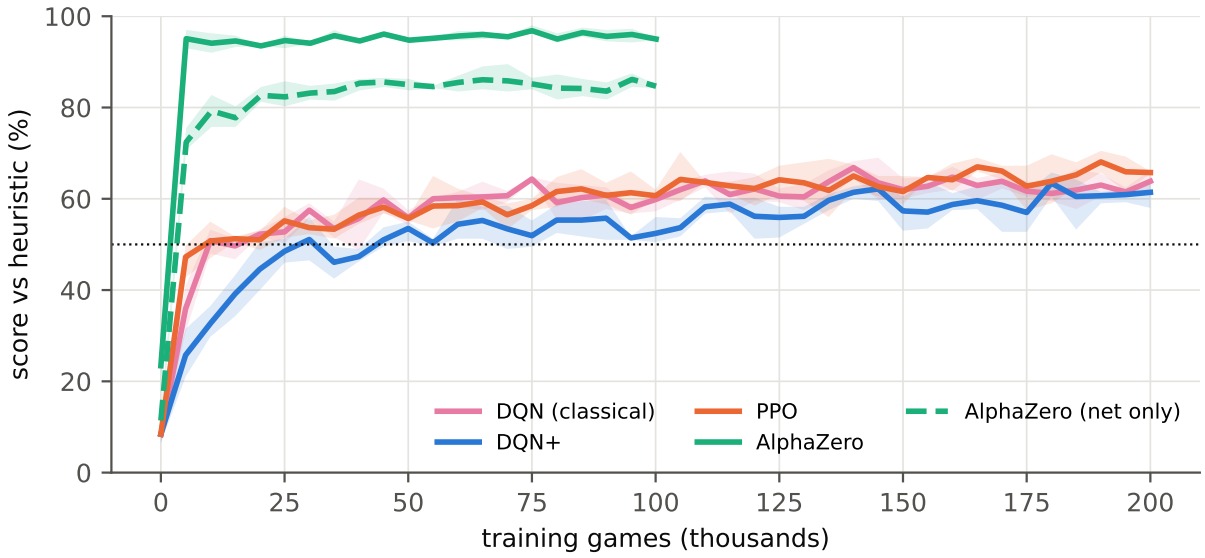


Figure 8: Learning curves: score against the heuristic bot during training (200 games per point; mean over 3 seeds, band: min–max). Only classical DQN trains against this bot; the others never see it during training.

Fig. 7 and Table 10 show a single, very large gap. DQN+ and PPO land at the level of classical DQN. AlphaZero is far above all of them on every opponent. It wins almost every game against the heuristic bot, and it is the first agent in this project that clearly beats minimax-d4.

Self-play without search: as good as the best fixed-opponent learner. DQN+ and PPO reach the strength of classical DQN without ever playing the heuristic bot, and PPO is slightly ahead on all three opponents. Self-play therefore removes the need for a hand-written training opponent at no cost in strength. It also removes the failure mode of §4: no self-play learner shows the "beats its training opponent but not random" pattern of Monte Carlo control. Their curves (Fig. 8) are still rising slowly at the end, as classical DQN's was. They level off near the same score, against heuristic and against minimax alike. We attribute this shared ceiling to what all three have in common: a one-step-lookahead policy computed by an MLP, with no search at play time. DQN+ learns more slowly than PPO. We hypothesise that the over-estimation fix and the opponent pool help less than PPO's direct policy updates when the reward is a single game result, but we have not tested this.

AlphaZero: two distinct contributions of search. The two AlphaZero bars separate two roles of search.

- **Search at play time.** The same network gains substantially, most of all against minimax-d4, when it is allowed to search 64 simulations per move. Search does at play time what minimax does: it looks at replies, which removes the horizon blunders that cap every one-ply policy.
- **Search as a training signal.** Even *without* search, the AlphaZero network is far stronger than the DQN+ or PPO networks. It draws level with minimax-d4 where they score about one in five. Its targets are what makes the difference. It is trained to imitate MCTS visit counts, a

policy already improved by lookahead, rather than to fit noisy game outcomes one move at a time.

One confound remains. The AlphaZero network is a convolutional residual network, while DQN+ and PPO use MLPs, so part of the second gap may come from the architecture's spatial inductive bias rather than from the training signal. Separating the two needs an AlphaZero run with a fully connected trunk, which we leave to future work.

Sample efficiency and compute. AlphaZero passes 80% against the heuristic bot by its first checkpoint, and its network alone does so within 15,000 games. DQN+ and PPO never reach 80%. Per game, however, AlphaZero is much more expensive: each move runs 64 network-guided simulations, and a run took several times longer in wall-clock time than a PPO run with twice as many games. The comparison still favours AlphaZero clearly. It reached 80% at its first checkpoint, after under twenty minutes of training, a level PPO did not reach in its full run.

Stability. Seed-to-seed spread is small for every method (Table 10), so the gaps between methods are far larger than the noise.

Limitation: the training curve saturates. AlphaZero's score against the heuristic bot levels off near 95% after roughly 20,000 games (Fig. 8). After that point the curve can no longer show progress. Minimax-d4, the only opponent still able to separate strong agents, was played only at the end, and intermediate checkpoints were not kept. We therefore cannot say whether AlphaZero was still improving in the second half of training. Future runs should save checkpoints and track a stronger yardstick, such as minimax at depth 6, throughout.

5.3 Comparison with the solved game

Table 11: The final model playing itself from the standard 6×6 start position, with greedy MCTS at three search budgets (3 games each, different seeds), compared with perfect play [9]. Results are Black–White disc counts.

Simulations	Games	Result(s)	Winner
64	3	15–21, 6–30	White
200	3	13–23	White
800	3	8–28	White
Perfect play	–	16–20	White

6×6 Othello is solved: with perfect play from the start, White wins 20–16 [9]. Our final model agrees on the *winner*. Its value head rates the start position at -0.63 from Black's side, a clear prediction that Black loses. In self-play from the start, White wins every game at every search budget (Table 11). It does *not* reproduce the perfect-play *margin*. White always wins by more than 4 discs. Perfect play guarantees Black at least 16, so a wider margin proves that Black is making mistakes. The model has learned which side is winning, but it does not play perfectly. Self-play was also mostly trained from random openings rather than from the start position, which may contribute.

5.4 The final model: Oroletto

The three AlphaZero seeds are within noise of each other. We chose the final model by its score against the strongest opponent, minimax-d4, since the weaker two opponents no longer separate the seeds. The selected run is `results/alphazero/seed1`, released as *Oroletto* through `models/oroletto.json`. Anyone can play it with `othello --play oroletto`, which by default searches 200 simulations per move, more than the 64 used in evaluation. We expect this to make it somewhat stronger than the scores reported here, but we have not measured it.

6 Does Opponent Variety Help DQN?

Self-play always faces an opponent of exactly the learner's own strength and style. A common alternative, used at scale by AlphaStar [23], is to train against a *league*: a population of different opponents, so that the learner meets many styles of play and cannot overfit to one. This section asks whether such variety helps DQN on 6×6. The test is a controlled, dose-response experiment: everything is held fixed except the share of training games played against the league.

6.1 Design

The league. The league consists of weak, fixed agents plus the learner's own past snapshots (Table 12). Each game is played against a league member with probability p, the *league share*, and against the live learner (pure self-play) otherwise. League games go half to a uniformly chosen fixed agent and half to a uniformly chosen snapshot, so the league's make-up stays the same throughout training (until the first snapshot exists, all league games go to fixed agents). We ran $p \in \{0, 0.25, 0.5, 0.75, 1\}$, with $p = 0$ being pure self-play. The learner is the Double DQN of §5, with identical network and hyperparameters: 200,000 games and 3 seeds per mix, so 15 runs in total.

Table 12: League members. "Strength" is each fixed agent's score against the heuristic bot from the earlier sections (§3, §4).

Member	Kind	Strength vs heuristic (%)
random	rule-based	9.5
heuristic	rule-based	–
minimax-d4	rule-based	87.5
q-learning	traditional RL (Phase 1)	10.2
sarsa	traditional RL (Phase 1)	8.0
mc-control	traditional RL (Phase 1)	56.1
linear	traditional RL (Phase 1)	54.0
dqn-classical	traditional RL (Phase 1)	65.1
own snapshots	frozen copies of the learner (every 5k games, ≤20)	varies

Hypotheses. Stated before the runs:

H1 (robustness):
variety makes DQN more robust, so it loses less to unfamiliar opponents.

H2 (weak signal):
once the weak members are beaten, games against them teach little, so league-only training ($p = 1$) ends up weaker than self-play.

H3 (best mix):
an intermediate mix keeps self-play's matched-strength opponent while adding variety, so held-out strength peaks at an intermediate p.

H4 (replay dilution):
at high p the replay buffer fills with games against weak opponents, which slows learning from the start.

Held-out evaluation. Heuristic and minimax-d4 are now *training* opponents. As Monte Carlo control showed in §4, a learner can score well against an opponent it trains on without having learned the game. "Better than self-play" is therefore judged on opponents the league learners never trained against, chosen to be close to DQN's own strength so that differences are visible:

Head-to-head vs self-play:
every model plays the three $p = 0$ models, 200 games per pair of seeds.

Minimax at depths 2 and 3:
not in the league. They share the heuristic's evaluation function, so they are *partly* related to two league members. A calibration run put Phase 2 DQN+ at 33% and 25% against them, so neither is too easy nor too hard.

Phase 2 PPO and DQN+:
completely unseen learned agents (3 seeds each, 200 games per pair).

In-league bots:
random, heuristic and minimax-d4 are reported too, but marked as training opponents.

All matches use paired colours and fixed random 6-ply openings, separate from those used in training. A mix *beats self-play* only if the seed-level bootstrap 95% CI of its score against the $p = 0$ models lies entirely above 50% *and* it is not worse than $p = 0$ against any held-out opponent. This rule was fixed before the runs.

6.2 Results

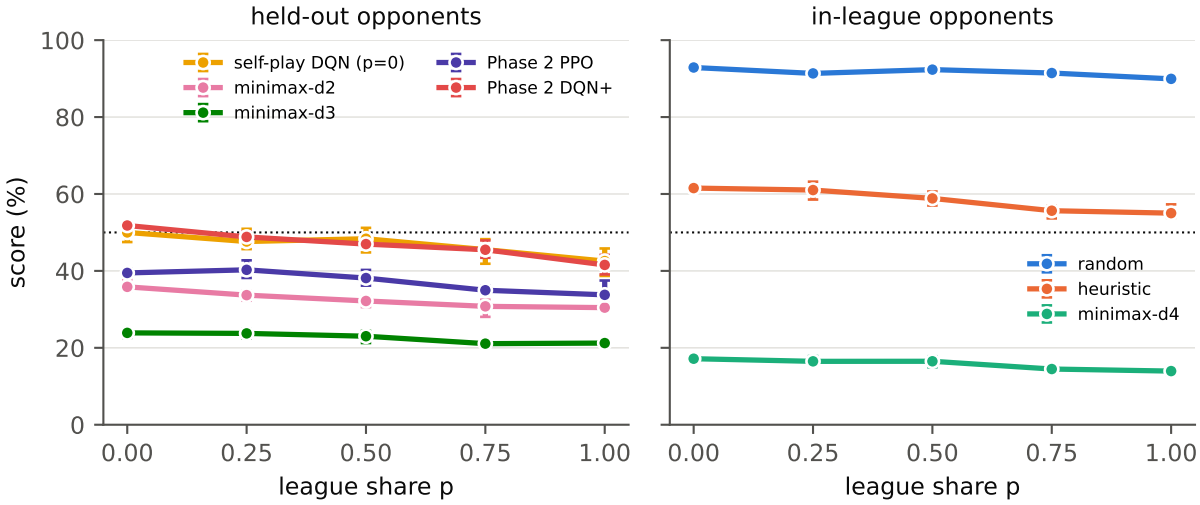


Figure 9: Final score against the league share p. Left: held-out opponents, including head-to-head against the self-play models (with bootstrap 95% CI). Right: league members (training opponents). Mean over 3 seeds; whiskers: min–max across seeds; dotted line: 50%.

Table 13: Phase 3 results (%). "vs $p = 0$" is the mean score against the three self-play models over all seed pairs, with a seed-level bootstrap 95% CI. The $p = 0$ row is self-play against the other self-play seeds, a sanity check that should read about 50. Other columns: mean ± std over 3 seeds. Bold: best per column.

	Held-out					In-league		
p	vs $p = 0$	minimax-d2	minimax-d3	Phase 2 PPO	Phase 2 DQN+	random	heuristic	minimax-d4
0	**50.0 [47.5, 52.5]**	**35.9 ± 1.7**	**23.9 ± 1.1**	39.5 ± 1.0	**51.8 ± 1.6**	**92.9 ± 0.3**	**61.5 ± 0.5**	**17.2 ± 0.8**
0.25	47.6 [45.7, 50.9]	33.7 ± 1.3	23.7 ± 1.2	**40.3 ± 2.3**	48.8 ± 1.7	91.3 ± 0.8	61.0 ± 2.3	16.5 ± 1.2
0.5	48.4 [44.8, 51.2]	32.2 ± 1.4	23.0 ± 1.6	38.1 ± 2.0	47.0 ± 0.6	92.3 ± 0.5	58.8 ± 1.8	16.5 ± 1.4
0.75	45.5 [41.9, 48.2]	30.8 ± 2.4	21.1 ± 0.9	35.0 ± 1.0	45.5 ± 2.3	91.5 ± 1.1	55.6 ± 1.7	14.5 ± 0.4
1	42.6 [38.7, 45.8]	30.4 ± 0.8	21.2 ± 0.5	33.8 ± 3.2	41.6 ± 2.5	89.9 ± 0.9	55.0 ± 2.0	13.9 ± 0.9

Fig. 9 and Table 13 show one clear trend: **no league share beats pure self-play, and DQN gets steadily weaker as the league share grows**. Against the self-play models, the low mixes ($p = 0.25, 0.5$) are statistically tied with self-play: their CIs include 50%. The high mixes ($p = 0.75$, 1) are significantly worse: their CIs lie entirely below 50%. The same downward slope appears against the held-out bots and the held-out learned agents. The head-to-head matrix between mixes (Fig. 10) tells the same story without involving any outside opponent: every lower-p mix scores at least level with, and mostly above, every higher-p mix it meets.

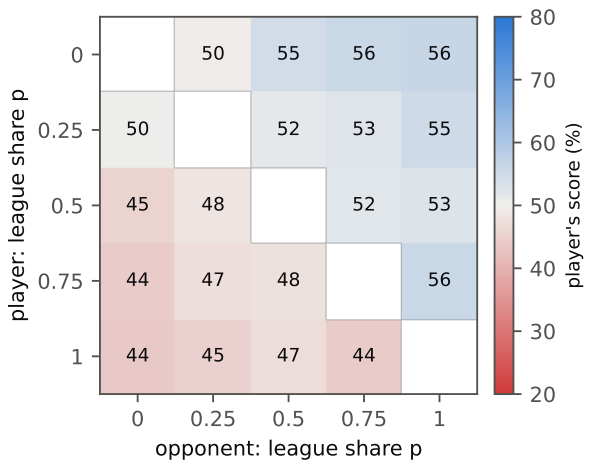


Figure 10: Head-to-head between mixes: the row mix's score against the column mix, over all 9 seed pairs (1,800 games per cell).

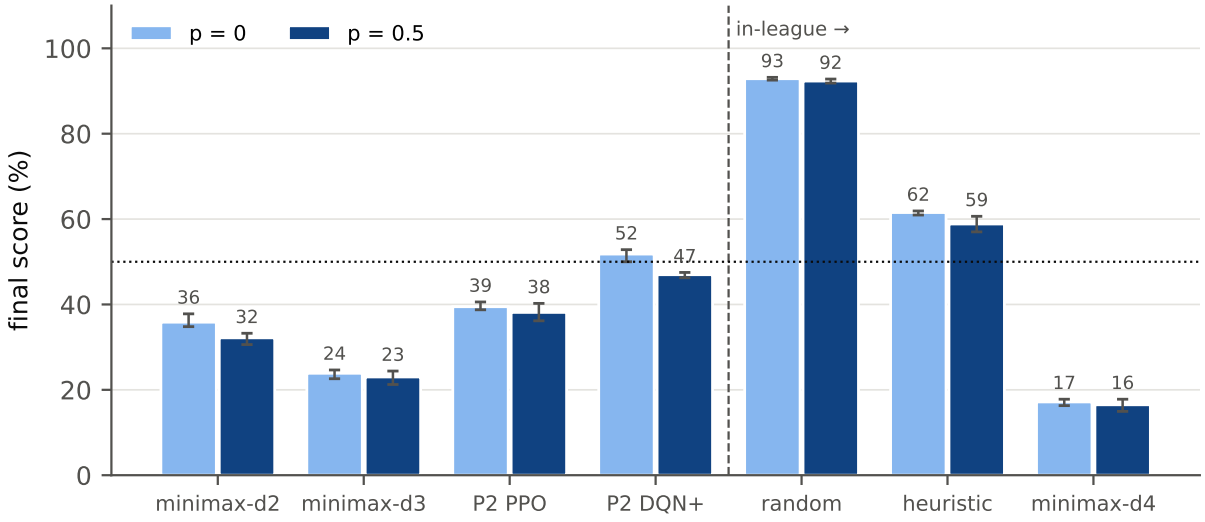


Figure 11: Self-play ($p = 0$) vs the best league mix ($p = 0.5$) against every panel opponent; the dashed line separates held-out from in-league opponents. Mean over seeds; whiskers: min–max.

Even the training opponents get harder to beat. The most striking result is on the right of Fig. 9. Scores against heuristic and minimax-d4 *fall* as p grows, even though the high-p learners actually play these bots during training. This contradicts our expectation that in-league scores would rise. The likely reason is a matter of dose. With uniform two-level sampling, a single fixed member such as the heuristic bot appears in only $p/16$ of all games, about 6% even at $p = 1$. That is far too little to memorise it, in contrast to Phase 1's Monte Carlo learner, which played the heuristic bot in every game. In-league scores therefore track the learner's general strength, and they fall with it. The random-bot score stays flat because every mix beats random almost every game.

Why variety hurts here: the training signal. Fig. 12 shows how the learner fared against each opponent during training. The league is far from uniformly informative. Half of the fixed members (random and the three tabular agents) are beaten about 90% of the time almost from the start, while minimax-d4 wins most of its games. Games that are nearly certain wins or nearly certain losses carry little information about *which* moves made the difference. Only the remaining members, and the learner's own snapshots, give evenly matched games. Self-play, by construction, gives evenly matched games *every* time: the opponent is always exactly as strong as the learner. Replacing self-play games with league games therefore lowers the share of informative games, and the learner improves more slowly. We attribute the downward trend mainly to this loss of matched-strength training signal. This supports H2, and its gradual, monotone shape explains why H3's interior peak never appears.

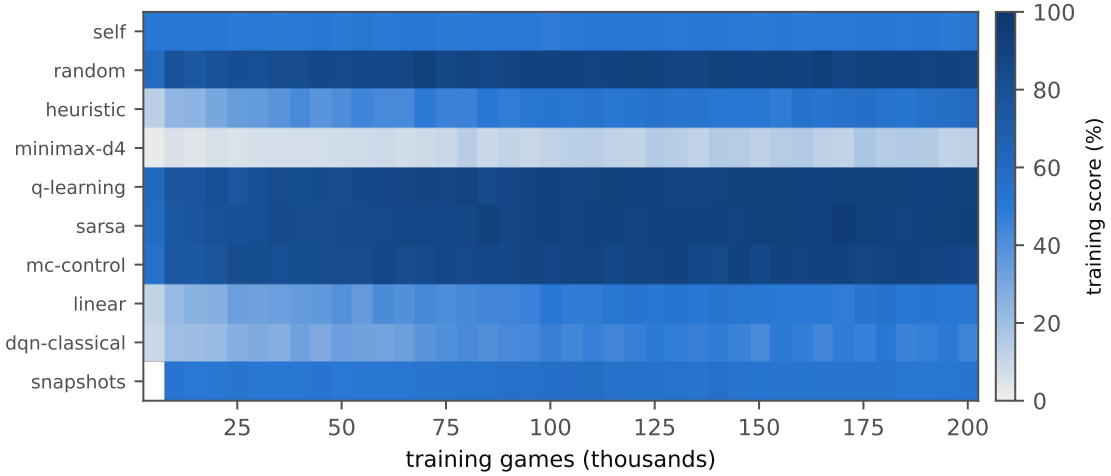


Figure 12: Training-time score against each opponent over training, for the $p = 0.5$ mix (mean over 3 seeds; exploring policy, so slightly below greedy strength). Dark: the learner wins almost always; light: it almost always loses.

Replay dilution (H4). DQN's replay buffer mirrors the mix: the share of league data in it equals p (Table 14). The checkpoint curves (Fig. 13) show the predicted early lag. League-only training starts behind against both held-out opponents. The gap narrows over training but does not close. This is *consistent with* dilution: positions reached against weak or idiosyncratic opponents are a skewed sample of the positions that matter against strong play, so values fitted to them transfer poorly. We cannot, however, separate dilution from the simpler explanation that high-p runs see fewer self-play games. Both predict the same curves, so H4 is only partly supported.

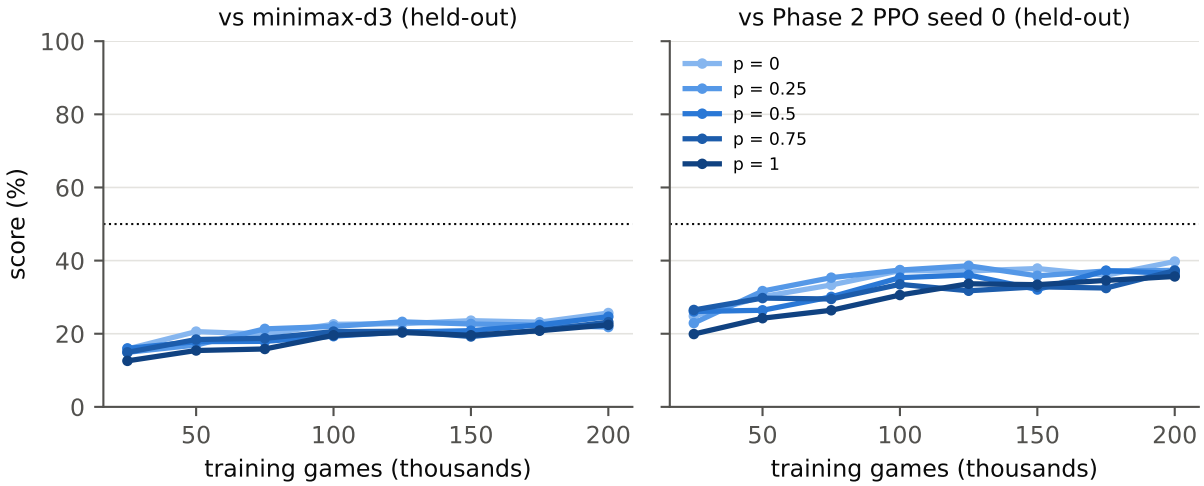


Figure 13: Held-out score of saved checkpoints during training, per mix (mean over 3 seeds; 200 games per checkpoint and seed). Lighter: lower league share.

Table 14: Replay-buffer composition and held-out learning speed. Buffer share: fraction of transitions in the replay buffer that came from league games at the end of training (mean over seeds). Scores: against minimax-d3 at 50k games and at the end (%).

p	Buffer share from league (%)	vs minimax-d3 at 50k	vs minimax-d3 at 200k
0	0	20.6	25.7
0.25	25	16.9	21.8
0.5	50	17.8	24.7
0.75	75	18.4	23.2
1	100	15.4	22.4

Self-play with or without past snapshots. The pure self-play learner here ($p = 0$) scores about evenly against Phase 2's DQN+, which played a pool of its own past snapshots half the time (Table 13). Playing old versions of itself made no measurable difference to DQN either, so for this learner the matched-strength live opponent is what matters.

Verdict on the hypotheses.

H1 (robustness): refuted.
League-trained DQN is no more robust. At low p it is level with self-play against the held-out opponents, and at higher p it is weaker against all of them, including the unseen learned agents.

H2 (weak signal): supported.
League-only training is significantly worse than self-play. The training-time scores show why: most league games are decided before they start.

H3 (best mix): refuted.
Strength falls monotonically with p. The low mixes merely tie with self-play.

H4 (replay dilution): partly supported.
There is an early lag that narrows over training, but it is confounded with the smaller amount of self-play data.

Implications and limitations. For DQN on 6×6, variety from *weak* opponents is not a substitute for matched-strength self-play. If a league is to help, its opponents need to be near the learner's own strength. That is exactly what AlphaStar's prioritised fictitious self-play (PFSP) arranges, by sampling the opponents the learner currently struggles against. It is the natural next variant, and the sampler is already built to swap it in. The conclusions are limited to this setting: a weak-only league, uniform sampling, one learner and 3 seeds per mix. The minimax-d2 and d3 opponents are only partly held out, but the fully unseen PPO and DQN+ agents show the same trend.

7 System Architecture

All experiments run on one Python code base (package `othello`). It has a size-parameterised environment, the bots, the learners, the evaluation harness, and a terminal front-end for human play.

7.1 Environment design

The environment is shared by every experiment, and each part of it exists for a downstream need:

Bitboard state.
A position is two integers of n^2 bits (one per colour), plus the side to move and a consecutive-pass counter. Move generation shifts the bitboards in the eight compass directions and masks out bits that wrap around a row edge. Nothing depends on a fixed n, so 6×6, 8×8 and 10×10 run the same code. This is the precondition for the size-generalisation study.

Immutable states.
Applying a move returns a new state rather than mutating the old one. Copying a position is therefore free, which tree search (minimax now, MCTS later) relies on to hold many positions at once.

Action space with an explicit pass.
Actions are the n^2 squares plus one *pass* action. Pass is legal only when no placement is. A fixed-size action space with a legality mask lets learned policies mask illegal moves rather than learn to avoid them.

Configurable input planes.
The state can be encoded as any subset of {own discs, opponent discs, empty squares, side to move, legal moves}. Any future input-representation experiment varies this set without touching the environment.

One agent interface.
Every player implements `act(state)`→`action`. Bots, learned policies and search agents are therefore interchangeable in the evaluation harness.

7.2 Evaluation protocol

All comparisons in this report use the same protocol. A fixed seed generates K opening positions by playing six uniformly random plies from the standard start. Each opening is played twice with colours swapped, which cancels the *first-move advantage*: the systematic edge (positive or negative) that comes from moving first rather than from playing well. Results are reported from the first agent's perspective as wins/draws/losses, *score* (win = 1, draw = $\frac{1}{2}$) with a 95% normal-approximation confidence interval, and mean final disc margin. The random openings keep a deterministic pair of agents from replaying one game $2K$ times. They also stop a learner from memorising a single line against a fixed opponent, which matters for the tabular methods in §4. Six plies give about 95% distinct openings at K=500, against about 42% for four plies. The normal-approximation interval degenerates (to zero width) when a score is exactly 0 or 1. Those cells should be read as "no losses observed", not as certainty.

7.3 Correctness validation

Table 15: Environment correctness checks (48 automated tests, all passing; see `results/baselines/pytest.txt`).

Check	What it guards against
Perft, 8×8, depth 1–6	Errors in move generation or flipping. Leaf counts must equal the published values 4, 12, 56, 244, 1396, 8200.
Naive cross-check, $n \in \{4, 6, 8, 10\}$	Edge-wrapping errors at any board size. On 30 random games per size, the legal-move set and flipped discs must match an independent 2-D grid implementation at every ply.
Forced pass / double pass	Wrong termination. A player with no move must pass; two consecutive passes end the game; a placement resets the pass counter.
Full board, draw	Wrong winner attribution on equal disc counts.
Transpose symmetry	Direction-specific shift bugs. Transposing the board must transpose the legal-move set.
Encoding shapes	Plane-construction errors. Every plane configuration has shape (C, n, n) with the expected disc and legal-move counts.
Agents	Illegal moves from any bot; minimax missing a forced win; the heuristic bot ignoring a corner.

Perft ("performance test") counts the leaf positions of the full game tree to a fixed depth. A single wrong flip or missed move changes the count, which makes it the most sensitive check of the move generator. Perft only exists for 8×8, so the naive cross-check carries the burden for the other sizes. It compares the bitboard engine with an independent implementation that shares no code with it, at every position of real games, including the board edges where wrap-around bugs would appear.

7.4 Simulator throughput

Table 16: Single-core simulator throughput, random vs. random, 1000 paired openings per size. "Black score" is the share of points won by the first mover under uniformly random play.

Board	Games	Games/s	Plies/s	Plies/game	Black score (%)
6×6	2000	**2,142**	56,910	26.6	46.9
8×8	2000	917	49,938	54.4	48.0
10×10	2000	449	40,597	90.4	47.8

Table 16 shows games per second falling steeply with board size while plies per second fall only mildly. Almost all of the slowdown therefore comes from games getting longer: the number of plies grows roughly with the number of squares. Only a small part comes from each move costing more. Cost per ply grows slowly because each bitboard operation acts on all n^2 squares at once and the directional scans take $\mathcal{O}(n)$ steps. This tells us how to budget compute. Simulation will be negligible next to neural-network inference in the self-play studies, and going from 6×6 to 10×10 costs mainly through game length. The pure-Python engine is therefore sufficient, and a compiled engine is not needed yet.

On every board size the first mover scores slightly below half under random play, so the side moving second has a small edge. This is *consistent with* the known result that 6×6 Othello is a second-player win under perfect play. Random play, however, says nothing about perfect play, so we read it only as a reason to keep evaluating with both colours.

7.5 Playing in the terminal

The `othello` command lets a person play any agent in the terminal: `othello --play [1--5 | beginner …expert]` chooses a rule-based bot, `othello --model PATH` loads a trained model (linear or DQN), `othello --play oroletto` plays *Oroletto*, the project's final trained model (§8), and `othello --tutorial` teaches the rules with the examples of Fig. 1 before a coached practice game. Options cover the board size (4–10), colour, legal-move hints and a coach mode that flags moves where the expert bot disagrees. The command reuses the same environment and agent classes as the experiments.

8 Next Steps

- **Prioritised league sampling.** Repeat the league experiment of §6 with PFSP, so that DQN mostly meets opponents near its own strength. This tests whether variety helps once the training signal is matched.
- **Exploiters and blind spots.** Train exploiter agents whose only goal is to beat a fixed model such as Oroletto. Then audit every agent's moves with an exact endgame solver to find and categorise its blind spots, and measure how far each model's weaknesses transfer to the others.
- **Multi-agent co-evolution.** Let several independent agents learn by playing one another, and compare the resulting models with the self-play models: their strength, how exploitable they are, and whether strategies cycle within the population.

- **Scaling to 8×8.** Re-tune the rule-based bots for 8×8 (Table 6 shows the 6×6 weights are weaker there) and train the selected configuration. Optionally, play matches against the Edax engine.
- **One network for every board size.** Test zero-shot transfer, then train a fully convolutional, globally pooled network on mixed board sizes and compare it with a small-to-large curriculum. The result is reported as a train-size × test-size win-rate matrix.

A The Heuristic Bot: Weight Tuning

This appendix documents how the heuristic bot's weights (Table 2) were chosen, and what the search reveals about the importance of each feature.

A.1 Grid search

The heuristic bot is not trained. Its four weights are chosen by a *grid search*: every combination of disc $\in \{0, 1\}$, mobility $\in \{0, 2, 5, 10\}$, corner $\in \{0, 10, 25, 50\}$ and X-square $\in \{0, -10, -25\}$ (96 candidates) plays 400 games, both colours, from random openings, against each of three fixed *tuning opponents*:

- **disc-greedy**: the same one-ply bot with all of its weight on disc count (disc = 1, all others 0), i.e. the naive "grab as many discs as possible" player;
- **hand default ("hand def.")**: the same one-ply bot with the weights we picked by hand *before* any tuning (disc 1, mobility 5, corner 25, no X-square term). It already values corners and mobility, so it is the strongest of the three tuning opponents and the one that separates good weightings from merely adequate ones;
- **random**: the floor.

A candidate is ranked by its *mean* score over the three opponents. Using a weak, a medium and a strong opponent together avoids selecting weights that only exploit one particular bot.

Table A.1 has three blocks.

Top block: the five best candidates.
Rank 1 is the selected configuration used everywhere else in this report.

Middle block: leave-one-out ablation.
Each row is the selected configuration with exactly *one* feature switched off: its weight is set to zero (shown in **red**) and the other three are kept. Comparing a row with Rank 1 shows how much that single feature contributes: the larger the drop in score, the more the bot relies on it. All four rows are points of the same grid, so no extra games were needed.

Bottom block: reference points.
The disc-count-only bot and the hand default, i.e. the naive strategy and our starting guess, scored by the same protocol.

Table A.1: Grid search over heuristic weights on 6×6 (96 candidates, 400 games per opponent, ±95% CI ≈ 4–5 points per cell). Scores (%) against the three tuning opponents: disc-greedy, the untuned hand default ("hand def.") and random. Top: best five candidates. Middle: the selected weights with one feature switched off (the zeroed weight in **red**). Bottom: reference bots. Bold: best shown per column.

	Weights				Score vs. (%)			
Configuration	disc	mob.	corner	X-sq.	disc-greedy	hand def.	random	mean
Rank 1 (selected)	1	5	50	-25	85.5	57.1	90.5	**77.7**
Rank 2	0	2	50	-25	85.1	56.6	90.6	77.5
Rank 3	1	5	50	-10	**87.1**	56.2	88.9	77.4
Rank 4	0	2	50	-10	85.6	55.2	90.2	77.0
Rank 5	1	2	50	-10	84.5	**58.9**	87.4	76.9
Selected, no disc	**0**	5	50	-25	83.6	57.4	88.8	76.6
Selected, no mobility	1	**0**	50	-25	82.2	47.0	88.2	72.5
Selected, no corner	1	5	**0**	-25	55.0	30.1	62.9	49.3
Selected, no X-square	1	5	50	**0**	84.4	50.4	**91.6**	75.5
Disc count only	1	0	0	0	48.5	16.6	59.0	41.4
Hand default	1	5	25	0	84.1	51.1	90.2	75.2

A.2 Feature importance

Table A.1 gives a clear ranking of feature importance. Corner occupancy dominates. Removing it hurts far more than removing any other feature and leaves a bot close to the disc-only player. Corners matter so much because of *stability*: a corner disc is permanent, and it lets the adjacent edge discs become permanent too. An evaluation that ignores corners optimises quantities that the opponent can reverse in one move.

Mobility is the second most important feature. Removing it hurts most against the hand-default bot, the opponent that itself values corners and mobility. We attribute this to mobility's *tempo* role. Restricting the opponent's options eventually forces them onto X- and C-squares that give up corners. This effect matters most against an opponent that already guards the corners, while weak opponents give corners away even without that pressure.

The disc-count weight barely matters, and the disc-only bot is the weakest configuration in the table, barely better than random. This is the classical *disc-count trap*. Grabbing material mid-game flips many discs that the opponent can flip back, and it increases the opponent's mobility. This result motivates the disc-count ablation for linear function approximation in §4.

The X-square penalty gives a smaller but consistent gain against the hand-default bot. That fits its role of refining corner play, since it only matters when the opponent is able to take corners.

Caveat. The top five configurations differ by less than the per-cell confidence interval. The ranking within that group is therefore noise-level. Selecting the best of 96 noisy candidates also inflates the selected configuration's score (a winner's-curse bias). The selected weights are a sound reference opponent but not a provably optimal one.

References

[1] L. V. Allis. *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis, University of Limburg, 1994.

[2] C. Berner et al. Dota 2 with large scale deep reinforcement learning. arXiv:1912.06680, 2019.

[3] M. Buro. Statistical feature combination for the evaluation of game positions. *Journal of Artificial Intelligence Research*, 3:373–382, 1995.

[4] M. Buro. The Othello match of the year: Takeshi Murakami vs. Logistello. *ICCA Journal*, 20(3):189–193, 1997.

[5] M. Campbell, A. J. Hoane, and F.-h. Hsu. Deep Blue. *Artificial Intelligence*, 134(1–2):57–83, 2002.

[6] Stanford University. CS 121, Handout No. 5, Homework 3: Game-Playing (Othello: the rules). Winter Quarter 1992. `https://stacks.stanford.edu/file/druid:bp488cx8853/bp488cx8853.pdf`.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[8] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *AAAI*, 2016.

[9] J. Feinstein. Amenor wins world 6×6 championships! *British Othello Federation Newsletter*, July 1993.

[10] D. E. Knuth and R. W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.

[11] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. In *ICLR*, 2023.

[12] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[13] P. S. Rosenbloom. A world-championship-level Othello program. *Artificial Intelligence*, 19(3):279–320, 1982.

[14] G. A. Rummery and M. Niranjan. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University, 1994.

[15] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.

[16] J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[17] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[18] D. Silver et al. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.

[19] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[20] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[21] H. Takizawa. Othello is solved. arXiv:2310.19387, 2023.

[22] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.

[23] O. Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575:350–354, 2019.

[24] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.